

Poisson Experiment

Berkay Köktürk
Boğaziçi University • Physics Department

Abstract—This report explores the applicability and behavior of the Poisson distribution in modeling radioactive decay events. Beginning with a historical overview, it explains the fundamental theory behind the Poisson model and its relevance to rare, discrete processes such as gamma-ray emissions. The experiment uses a Geiger-Müller tube setup to collect count data from different gamma-ray sources under various time intervals. The collected data are analyzed through histograms and fitted with Gaussian and Poisson distributions. Key parameters such as mean, standard deviation, and χ^2/nfd values are compared to evaluate the fit quality[1]. Additionally, special cases of the Poisson distribution involving time-dependent probabilities are studied, leading to an experimental estimation of the decay rate parameter α . The findings confirm that Poisson fits outperform Gaussian fits when dealing with low-count rare events.

I. THEORY

A. HISTORY

Poisson distribution was first introduced by the French mathematician *Siméon Denis Poisson* in 1837[2]. It was originally designed to explain the probability (distribution) of rare events occurring over a fixed time interval. Poisson first published the distribution in his work "*Recherches sur la Probabilité des Jugements en Matière Criminelle et en Matière Civile*".

The distribution has gained a wider attention after it is tested on the number of soldiers died from horse kick in Prussia. The empirical results were first documented by Ladislaus Bortkiewicz in 1898[3].

In the 20th century, due to its great success in explaining the distribution of rare random events, the Poisson distribution is mostly used in the areas of nuclear physics, telecommunication, insurance mathematics.

The distribution is especially relevant for explaining the radioactive decay events as they are rare and random.

B. THEORY

In this experiment, we are dealing with the distribution of the rare events which are radioactive emission. Thus, we will use the Poisson distribution.

Poisson distribution is given as follows:

$$P(k; \lambda) = \frac{\lambda^k e^{-\lambda}}{k!} \quad (1)$$

And therefore, we want it to be normalized to unity.

$$\sum_k P(k; \lambda) = 1 \quad (2)$$

Maybe the most famous equation about the Poisson distribution is the following[4, 1]:

$$\sigma^2 = \mu \quad (3)$$

At this point, we need to find what μ and σ are.

$$E[X] = \sum_{x=0}^{\infty} x \cdot \frac{\lambda^x e^{-\lambda}}{x!} \quad (4)$$

$$= \sum_{x=1}^{\infty} x \cdot \frac{\lambda^x e^{-\lambda}}{x!} \quad (5)$$

$$= \lambda \sum_{x=1}^{\infty} \frac{\lambda^{x-1} e^{-\lambda}}{(x-1)!} \quad (6)$$

$$= \lambda \sum_{x=0}^{\infty} \frac{\lambda^x e^{-\lambda}}{x!} \quad (7)$$

$$= \lambda \quad (8)$$

In line 7, we use eq.2 and it gives line 8. Since $E[X] = \mu$, we find that $\lambda = \mu$. Now, it comes to find that Eq.3 is true. The standard deviation of a distribution is given as:

$$\sigma = \sqrt{E[(X - \mu)^2]} \quad (9)$$

To simplify the RHS of Eq.9 in terms of λ we need to look at the second moment of the Poisson distribution[4]:

$$E[X^2] = \sum_{x=0}^{\infty} x^2 \cdot \frac{\lambda^x e^{-\lambda}}{x!} \quad (10)$$

$$= \sum_{x=1}^{\infty} x^2 \cdot \frac{\lambda^x e^{-\lambda}}{x!} \quad (11)$$

$$= \lambda \sum_{x=1}^{\infty} x \cdot \frac{\lambda^{x-1} e^{-\lambda}}{(x-1)!} \quad (12)$$

$$= \lambda \sum_{x=0}^{\infty} (x+1) \cdot \frac{\lambda^x e^{-\lambda}}{x!} \quad (13)$$

$$= \lambda(E[X] + 1) \quad (14)$$

$$= \lambda(\lambda + 1) \quad (15)$$

With the help of Eq.15, Eq.9 becomes the following:

$$\sigma = \sqrt{E[(X - \mu)^2]} \quad (16)$$

$$= \sqrt{E[X^2 - 2\mu X + \mu^2]} \quad (17)$$

$$= \sqrt{E[X^2] - 2\mu E[X] + \mu^2 E[1]} \quad (18)$$

$$= \sqrt{\mu(\mu + 1) - 2\mu^2 + \mu^2} \quad (19)$$

$$= \sqrt{\lambda} \quad (20)$$

The another fact that we will be intriguing in this experiment is that the Poisson and Gaussian distributions are almost indistinguishable when the mean μ is large.

Normalized Gaussian distribution is given as:

$$G(x; \mu, \sigma) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left[-\frac{(x - \mu)^2}{2\sigma^2}\right] \quad (21)$$

On the other hand, we will examine the other property of the Poisson distribution. When it comes to the distribution of successive events, the Poisson distribution shows a different behavior.

Since, now, we are interested in time intervals, let us denote $\mu = \alpha t$, then:

$$P(x; \mu) = P(x; \alpha, t) = \frac{(\alpha t)^x e^{-\alpha t}}{x!} \quad (22)$$

Then, the probability of having 1 event between the time interval dt becomes:

$$P(1; \alpha, dt) = \frac{(\alpha dt) e^{-\alpha dt}}{1!} \quad (23)$$

Having n successive event in t interval and 1 more successive event in dt interval can be written as the following[5, 6]:

$$P(x + 1; t)dt = P(x; \alpha, t)P(1; \alpha, dt) \quad (24)$$

$$= \frac{(\alpha t)^x e^{-(\alpha t)}}{x!} \frac{(\alpha dt) e^{-(\alpha dt)}}{1!} \quad (25)$$

$$= \frac{\alpha (\alpha t)^x e^{-(\alpha t)}}{x!} dt \quad (26)$$

In line 25, we assumed that $e^{-(\alpha dt)} \approx 1$ because $dt \approx 0$. Furthermore, line 26 allows us to say for $x = 0$, $P(1; t) = \alpha e^{-(\alpha t)}$ and for $x = 1$, $P(2; t) = \alpha (\alpha t) e^{-(\alpha t)}$. Hence, in the second part of the analysis, we will use these two forms of the Poisson distribution.

II. THE EXPERIMENTAL SETUP

The name of experimental devices and their purpose of use are listed below.

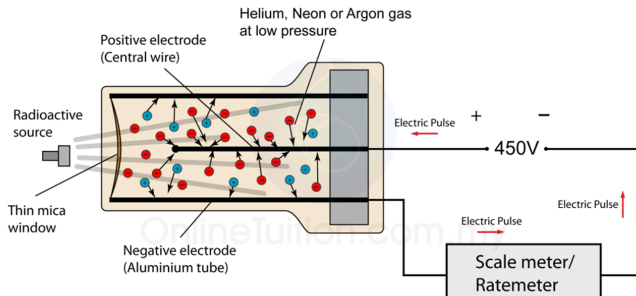


Fig. 1. Schematic Drawing of the Interior of Geiger-Muller Tube[7]

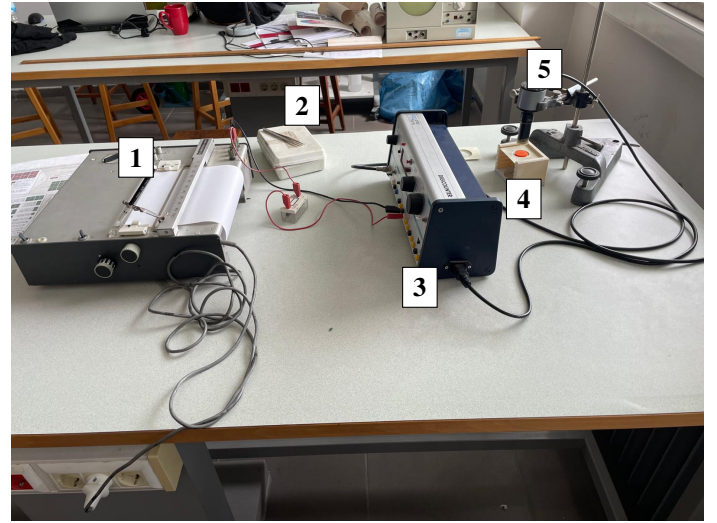


Fig. 2. Poisson Experiment Setup

- 1) **Chart Recorder:** It is an analog device that records the peaks (that correspond to successive hits of gamma rays) obtained from the *Geiger-Muller Tube* (no.5 in Fig.2).
- 2) **Sample Box:** It contains the various gamma ray sources (*Cs-137*, *Ba-133*, etc.).
- 3) **Counter (Scalar):** It is connected to the *Geiger-Muller Tube*. It is a digital device that counts every successive hit of gamma rays. It can be adjusted to different modes and different time intervals. We are going to use 100, 10 and 1 s intervals in this experiment.
- 4) **Sample Holder:** It holds one of the gamma ray sources at a certain distance from the *Geiger-Muller Tube*. The distance level of the *Sample Holder* can be adjusted. This adjustment will be useful later in the experiment.
- 5) **Geiger-Muller Tube:** It is the main device of the experiment. It basically creates a signal after a successive hit of a gamma ray. It consists of *anode*, *cathode*, *gas mixture*, and *mica window*. The scheme of the G-M tube can be found in Fig.1.
 - **Anode:** It is a thin conducting metal stick that is positively charged. It is placed in the middle of the tube.
 - **Cathode:** It is a negatively charged metal cover. It usually consists of metal/glass-type material and covers the outermost surface of the tube.
 - **Gas Mixture:** Generally, noble gases such as neon and argon are used. The incoming radiative rays face the gas mixture and start ionization processes. Ionized gases give free electrons and contribute to the chain reaction of ionization (which we call the avalanche effect). A high amount of free electrons rush to the anode, and they create a reasonable current in the system. Then G-M Tube counts this current as one successive hit. The gas mixture may contain a quencher to stop the avalanche effect.
 - **Mica Window:** It is just a thin window that is used to cover the entrance of the tube. It is used because of its low mass/area ratio.

Further information look at [5].

III. METHOD

We will briefly explain the mechanism working behind the experiment and then state the conducting procedure step-by-step.

First, we create the voltage difference between the anode and cathode ends of the G-M tube. This potential difference allows ionized electrons to accelerate to the anode end in the middle of the tube. If the potential is not high enough, we cannot observe the current in the G-M tube, thus counter does not count any successive hit of a gamma ray[8].

After choosing the operating voltage wisely, we placed a radioactive material on the sample holder and located it under the G-M tube. The material we put emits gamma rays. These emitted gamma rays may cross the mica window of the G-M tube, and hence they may travel along the gas mixture inside of the G-M tube. While a gamma ray is traveling in this mixture, it faces noble gases and makes them ionized. Ionized gases leave an electron behind them. This electron accelerates to the anode end and creates a chain reaction of ionization on its way. The avalanche of electrons allowed us to determine a reasonable amount of current between the ends of the G-M tube. This whole process is counted as one successive hit.

The procedure for conducting the experiment is given below.

- 1) Choose a gamma-ray source (when we were conducting the experiment, we used Cs-137) and place it on the sample holder (**no.4 in Fig.2**).
- 2) Adjust the counter (**no.3 in Fig.2**) at *radioactive, single, 100 s*.
- 3) Adjust the HV (high voltage) value to its minimum. Then start the counter and wait 100 seconds to complete its cycle. Under some critical value of HV, you get no data from the counter.
- 4) Increase the HV by 20V after each cycle and try to determine the plateau region of the G-M tube.
- 5) Choose an operating voltage that falls in the plateau region.
- 6) After this point, you will start collecting the data. Choose a gamma-ray source (we again chose the Cs-137) and place it on the sample holder and adjust the high of the tray so that you get about 100 counts while the counter is operating 10 s cycles.
- 7) Collect 100 data points as the counter is set to 10 s intervals.
- 8) By keeping the tray position still, repeat the previous step as the counter is set to 1 s intervals.
- 9) Choose another gamma-ray source, different tray position, and try to get 20-30 counts per 10 s long cycles.
- 10) Repeat steps 7 and 8.
- 11) Now, you have 4 data sets consisting of 100 data points. Find the source and tray position that result in 1 successive count per second.
- 12) Adjust the chart recorders (**no.1 in Fig.2**) speed to 12 inches per minute. Run for about 2 minutes.

You will get 4 data sets and a paper sheet that are necessary to complete the analysis after following the steps as in the order above.

Further information look at [5].

IV. THE DATA

We will share our data sets in this section. We conducted the experiment by using Cs-137 and Ba-133. We get 2 data sets for each of them that are for 10 s and 1 s intervals. We will give them in one table below. In addition, we share the measurements that we made on the sheet that we obtained from the chart recorder.

Data tables can be reached in Appendix A. We decide to put them in Appendix to not disrupt the readability of the paper. (TABLE VI, VII, VIII)

V. THE ANALYSIS AND RESULTS

As we explained in the method part, we first tried to find the plateau region of the G-M tube to choose a suitable operating voltage. You can see our test and data in TABLE VI. Here, we also post the graph of the Counts vs. Voltage.

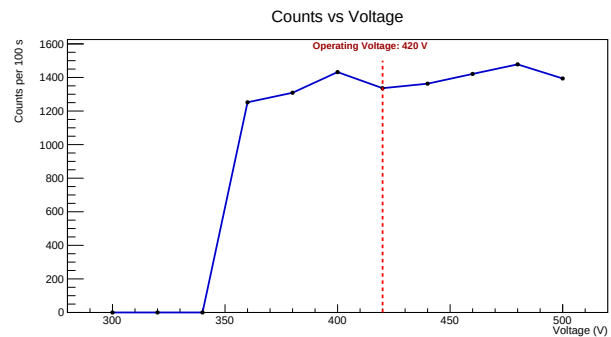


Fig. 3. Graph of Counts vs. Voltage

The plateau region should be more linear. Despite the fact that 400V is anomalous here in the graph, we choose our operating voltage as 420V.

A. PART 1

At the first part of the analysis we share the histograms created with respect to the TABLES VII and VIII. We will state the mean and standard deviations of the histograms. Then in separate graphs, we share the Gaussian and Poisson fits and their mean and standard deviations.

Fig. 4. Histogram of Cs-137 (10s) in TABLE VII and VIII

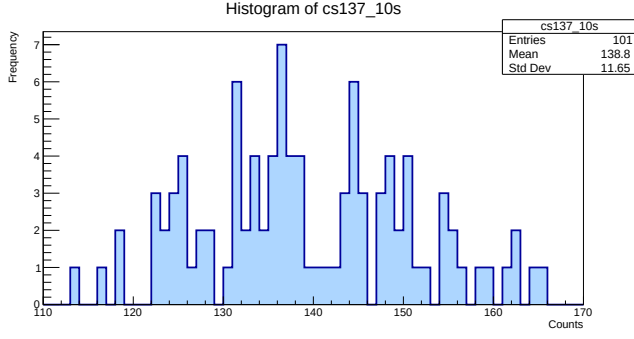


Fig. 6. Histogram of Cs-137 (1s) in TABLE VII and VIII

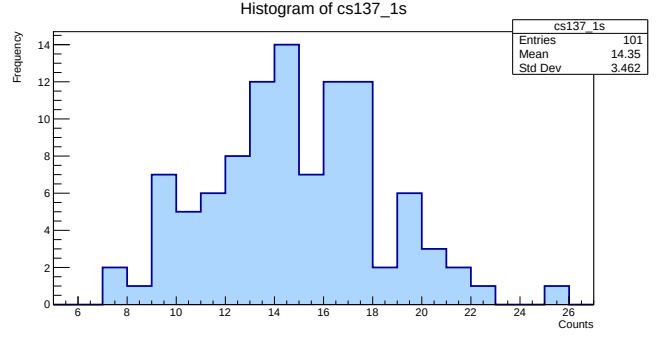


Fig. 5. Gaussian and Poisson fits of Cs-137 (10s) in TABLE VII and VIII

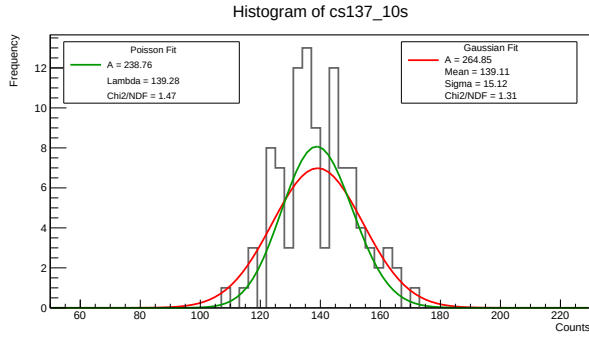


Fig. 7. Gaussian and Poisson fits of Cs-137 (1s) in TABLE VII and VIII

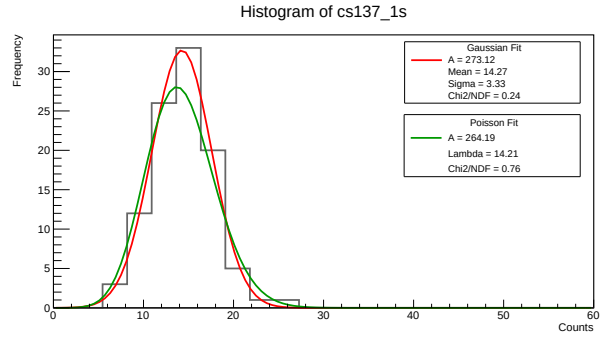


TABLE I
MEAN, STD. DEV., AMPLITUDE AND χ^2/ndf VALUES FOR FIG.4.5

	Gaussian	Poisson	Histogram
Mean (μ) (or λ)	139.11	139.28	138.8
Std Dev. (σ)	15.12	11.80	11.65
Amplitude (A)	264.85	238.76	-
χ^2/ndf	1.31	1.47	-

TABLE II
MEAN, STD. DEV., AMPLITUDE AND χ^2/ndf VALUES FOR FIG.6.7

	Gaussian	Poisson	Histogram
Mean (μ) (or λ)	14.27	14.21	14.35
Std Dev. (σ)	3.33	3.77	3.462
Amplitude (A)	273.12	264.19	-
χ^2/ndf	0.24	0.76	-

Fig. 8. Histogram of Ba-133 (10s) in TABLE VII and VIII

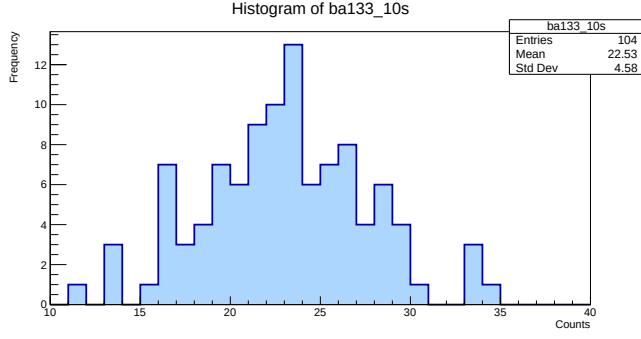


Fig. 10. Histogram of Ba-133 (1s) in TABLE VII and VIII

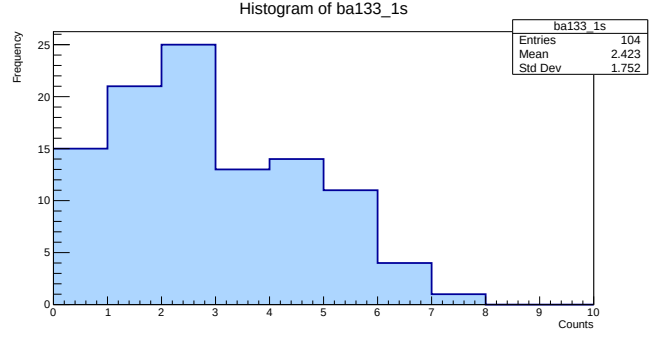


Fig. 9. Gaussian and Poisson fits of Ba-133 (10s) in TABLE VII and VIII

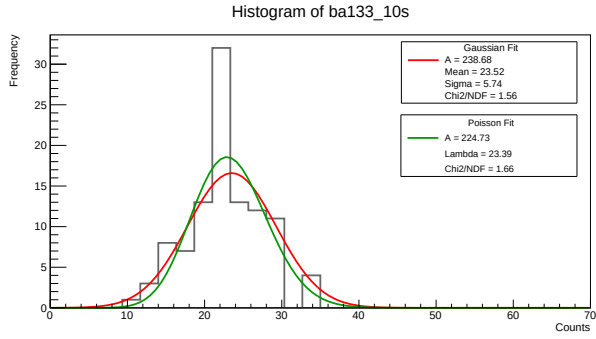


Fig. 11. Gaussian and Poisson fits of Ba-133 (1s) in TABLE VII and VIII

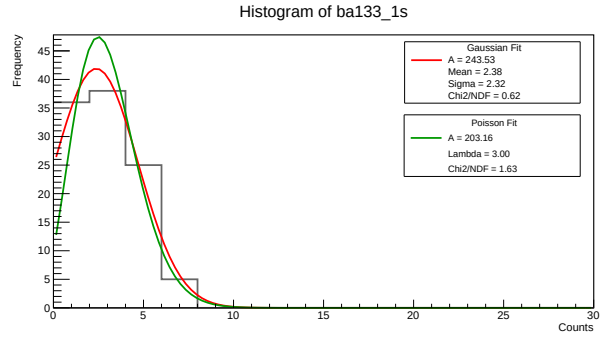


TABLE III
MEAN, STD. DEV., AMPLITUDE AND χ^2/ndf VALUES FOR FIG.8.9

	Gaussian	Poisson	Histogram
Mean (μ) (or λ)	23.52	23.39	22.53
Std Dev. (σ)	5.74	4.84	4.58
Amplitude (A)	238.68	224.73	-
χ^2/ndf	1.56	1.66	-

TABLE IV
MEAN, STD. DEV., AMPLITUDE AND χ^2/ndf VALUES FOR FIG.10.11

	Gaussian	Poisson	Histogram
Mean (μ) (or λ)	2.38	3.00	2.423
Std Dev. (σ)	2.32	1.73	1.752
Amplitude (A)	243.53	203.16	-
χ^2/ndf	0.62	1.63	-

Lastly, for this part of the analysis, we also want to show the $\sqrt{\mu}/\sigma$ ratio. If the Poisson distributions fit well to the histograms, then this ratio should be ≈ 1 .

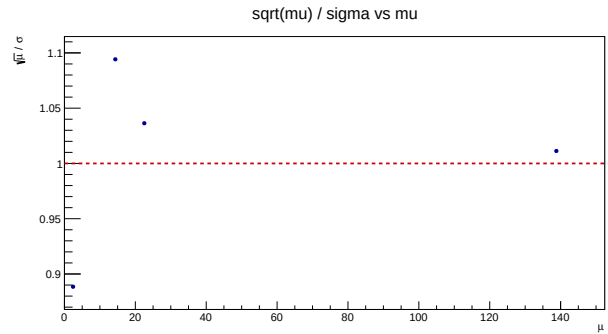


Fig. 12. $\sqrt{\mu}/\sigma$ ratio vs. μ graph for the histograms in Fig.4 6 8 10

B. PART II

In this part of the analysis, we will use the following two formulas that we have already derived in the Theory Part:

$$P_0(t) = P(1; t) = \alpha e^{-\alpha t} \quad (27)$$

$$P_1(t) = P(2; t) = \alpha(\alpha t) e^{-\alpha t} \quad (28)$$

We will use the TABLE IX and X and extract the value of α . Then, we will make two histograms for each of the tables and make Poisson fits as in Eq.27 and Eq.28. These fits will give us the experimental value of α .

Calculate the value of α :

$$\alpha = \frac{\text{number of peaks}}{\text{total time elapsed}} = \frac{226}{1769} = 0.128 \quad (29)$$

The histograms and corresponding Poisson fits are given below:

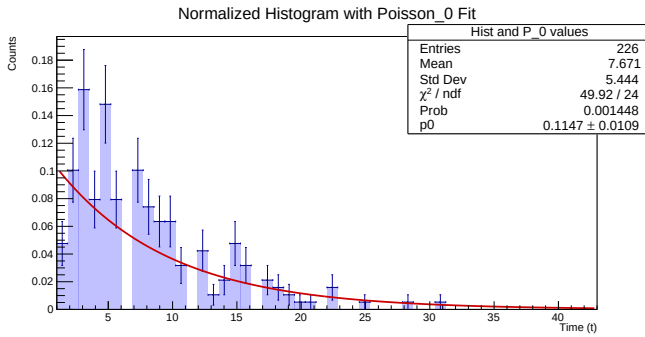


Fig. 13. Histogram filled with data in TABLE IX and $P_0(t)$ fit in Eq.27

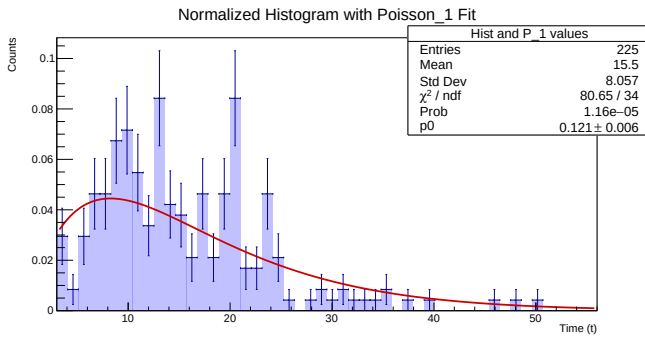


Fig. 14. Histogram filled with data in TABLE X and $P_1(t)$ fit in Eq.28

TABLE V
 α PARAMETER AND χ^2/ndf VALUES FOR FITS P_0 AND P_1

	$P_0(t)$	$P_1(t)$
α parameter	0.115 ± 0.011	0.121 ± 0.006
χ^2/ndf	2.08	2.37

VI. THE CONCLUSION

We will declare the results we found in the analysis part. We will try to make a comparison between goodness of the Gaussian and Poisson fits. Since the analysis part consists of two parts, we divide the conclusion part into two pieces.

A. PART I

Here, we will share the results of the analysis **PART I**.

First, we should observe Figs.5 and 9. In these two graphs, the mean values (μ) of the histograms are relatively high. It can be easily seen that the χ^2/ndf values are very close to each other, hence they are also close to the value 1. This implies that both fits are suitable to the data sets. Furthermore, it proves the fact that if the mean value is high within the data set, we cannot see a difference between the Gaussian and Poisson distributions.

However, when we look at the Figs.7 and 11, the mean values are low within the data set, thus we really see a difference between two distributions. In both cases, the χ^2/ndf values of the Gaussian fits are less than 1. This may seem good, but it actually tells us that the distribution function makes an overfit. The situation occurs for the Gaussian in both cases because the Gaussian distribution needs more data and it gives more reliable output when it operates at high values of mean. In conclusion, we can assure that, in the case of rare events, the Poisson distribution works really better.

B. PART II

Here, we will share the results of the analysis **PART II**.

In the second part of the analysis, we tried to observe the reliability of the special cases of the Poisson distributions given in Eqs.27 and 28. As calculated in Eq.29, expected value of α is $\alpha_{\text{expected}} = 0.128$.

Hence, after applying the fittings, we obtained the α values for the P_0 and P_1 distributions and they seem very close to the value we expected.

Experimental Value of α for P_0 : 0.115 ± 0.011

Experimental Value of α for P_1 : 0.121 ± 0.006

For both of the distributions, the expected value is 2σ away from the experimental values. This is highly accurate.

However, another thing that draws our interest is the χ^2/ndf values, which are >2 . This indicates that the distributions are not well-fitted. This is either due to the low amount of data number or to some unknown bias in the measurement devices.

VII. APPENDIX

A. DATA TABLES

TABLE VI
COUNTS(100s) VS. VOLTAGE (CS-137)

Voltage (V)	Counts(100s)
300	0
320	0
340	0
360	1252
380	1309
400	1432
420	1336
440	1363
460	1421
480	1478
500	1394

TABLE VII
DATA SETS OF CS-137 AND BA-133 FOR 10 AND 1 s INTERVALS.

No.	Cs-137 (10s)	Cs-137 (1s)	Ba-133 (10s)	Ba-133 (1s)
1	124	14	25	4
2	145	13	28	4
3	131	16	23	1
4	136	13	28	3
5	144	11	23	2
6	147	7	25	2
7	135	12	19	1
8	128	9	17	5
9	133	17	15	0
10	122	10	26	2
11	147	14	24	3
12	134	15	17	5
13	136	19	11	3
14	150	10	20	4
15	127	15	29	1
16	136	15	21	3
17	137	9	16	1
18	154	16	26	1
19	136	10	22	5
20	148	17	23	2
21	151	13	23	2
22	149	13	33	2
23	135	16	16	1
24	122	21	19	0
25	143	19	21	0
26	127	21	18	1
27	131	18	18	0
28	150	11	20	2
29	144	25	22	0
30	116	12	33	4
31	150	13	16	2
32	135	17	16	1
33	139	16	21	3
34	134	13	24	2
35	148	13	23	4
36	145	17	18	1
37	125	14	21	2
38	126	9	13	4
39	138	17	23	4
40	161	14	23	5
41	131	16	22	1
42	143	16	16	2
43	137	19	19	4
44	149	12	27	0
45	165	20	25	0
46	132	14	22	2
47	125	14	19	2
48	141	8	29	5
49	144	11	28	1
50	143	14	23	1
51	138	14	20	2
52	137	14	19	3

TABLE VIII
DATA SETS OF CS-137 AND BA-133 FOR 10 AND 1 s INTERVALS.
(CONTINUED)

No.	Cs-137 (10s)	Cs-137 (1s)	Ba-133 (10s)	Ba-133 (1s)
53	118	9	22	2
54	147	17	24	3
55	124	11	28	0
56	164	20	16	2
57	123	15	26	1
58	162	15	25	2
59	162	11	33	6
60	170	17	25	3
61	148	9	19	4
62	144	14	23	5
63	144	12	18	2
64	140	10	28	2
65	145	17	26	6
66	136	19	22	0
67	113	13	24	1
68	135	16	26	3
69	158	12	27	0
70	138	12	24	3
71	154	19	21	0
72	131	17	26	1
73	131	16	34	1
74	131	14	20	3
75	155	7	21	2
76	136	14	29	4
77	155	14	25	4
78	136	13	13	2
79	154	15	22	1
80	138	16	23	2
81	128	11	20	5
82	142	17	28	7
83	122	16	27	0
84	144	22	23	2
85	125	10	20	6
86	133	17	21	0
87	159	15	21	5
88	133	14	27	1
89	152	17	22	4
90	124	13	26	0
91	133	20	22	3
92	156	12	24	5
93	123	9	21	2
94	109	18	23	5
95	125	19	17	1
96	130	12	25	1
97	150	16	13	3
98	118	9	26	1
99	137	13	30	2
100	148	16	29	5
101	132	13	16	4
102			19	6
103			22	0
104			23	4

TABLE IX

DISTANCE(MM) BETWEEN THE PEAKS IN CHART RECORDER FOR N=0
CASE

Col 1	Col 2	Col 3	Col 4	Col 5
4	12	1	9	2
5	3	3	4	8
6	15	8	2	1
3	6	5	2	11
5	2	5	5	17
2	7	3	9	7
21	7	7	1	18
3	4	3	7	15
3	2	3	2	2
10	10	7	1	17
5	14	17	9	3
16	9	3	5	1
3	5	8	1	9
14	6	12	7	15
7	3	7	2	2
14	5	2	13	5
17	18	5	3	15
4	3	6	4	25
3	7	5	22	10
16	6	9	3	2
2	3	10	6	14
12	28	8	18	9
8	22	9	6	5
12	7	10	3	7
10	10	9	15	5
15	4	6	10	12
10	12	7	19	12
3	5	15	2	22
3	10	4	11	8
15	11	3	10	1
31	13	5	5	19
6	43	12	5	2
16	5	9	5	3
6	5	8	8	4
9	5	3	11	8
3	5	7	2	5
4	6	6	3	8
20	8	3	10	7
15	16	16	4	2
4	4	16	6	1
5	7	5	2	3
8	1	8	11	6
5	7	3	4	
8	4	3	7	
2	4	9	6	
5	3	7	11	

TABLE X

DISTANCE(MM) BETWEEN THE PEAKS IN CHART RECORDER FOR N=1
CASE

Col 1	Col 2	Col 3	Col 4	Col 5
9	17	7	12	11
11	15	4	16	13
9	18	4	16	17
8	21	11	13	13
7	8	13	6	10
23	9	10	4	9
24	14	8	7	12
6	11	10	14	28
13	6	10	10	24
15	12	6	8	25
21	24	10	9	33
19	23	24	3	17
17	14	20	10	19
21	11	11	14	20
21	9	20	6	4
31	8	19	8	10
21	23	9	9	24
7	21	7	15	17
19	10	11	16	7
18	13	11	7	20
14	9	14	26	40
20	31	19	25	35
20	50	18	9	12
22	29	17	24	16
25	17	19	24	23
25	14	19	9	14
13	16	15	18	12
6	17	13	25	12
18	15	22	29	17
46	21	19	21	24
37	24	7	13	34
22	56	8	21	30
22	48	17	15	9
15	10	21	10	20
12	10	17	10	21
7	10	11	13	5
24	11	10	19	7
35	14	13	13	12
19	24	9	5	13
9	20	19	13	13
13	11	32	14	15
13	8	21	10	9
13	8	13	8	3
10	11	11	13	4
7	8	6	15	9

B. ROOT CODES

```

void voltage_vs_counts() {
    const int n = 11;
    double voltage[n] = { 300, 320, 340, 360,
        380, 400, 420, 440, 460, 480, 500 };
    double counts[n] = { 0, 0, 0, 1252, 1309,
        1432, 1336, 1363, 1421, 1478, 1394 };

    TGraph* graph = new TGraph(n, voltage,
        counts);
    graph->SetTitle("Counts vs Voltage;Voltage
        (V);Counts per 100 s");
    graph->SetMarkerStyle(20);
    graph->SetMarkerSize(1.2);
    graph->SetLineColor(kBlue + 1);
    graph->SetLineWidth(2);

    TCanvas* c1 = new TCanvas("c1", "Voltage vs
        Counts", 800, 600);
    graph->Draw("APL");

    TLine* line = new TLine(420, 0, 420, 1500);

```

```

line->SetLineColor(kRed);
line->SetLineStyle(2);
line->SetLineWidth(2);
line->Draw("same");

TLatex latex;
latex.SetTextSize(0.03);
latex.SetTextColor(kRed + 2);
latex.DrawLatex(425, 1450, "Operating
    Voltage: 420 V");

c1->Update();
}

#include <TCanvas.h>
#include <TH1F.h>
#include <TF1.h>
#include <TMath.h>
#include <TStyle.h>
#include <fstream>
#include <vector>
#include <iostream>
#include <algorithm>

double pois_1(double* t, double* par) {
    double alpha = par[0];
    return alpha * alpha * t[0] *
        TMath::Exp(-alpha * t[0]);
}

void fit_normalized_histogram_1() {
    gStyle->SetOptFit(1111);

    const char* filename =
        "D:/root_v6.24.06/macros/y_i_values.txt";
    std::ifstream infile(filename);

    if (!infile.is_open()) {
        std::cerr << "ERROR: Cannot open file "
            << filename << std::endl;
        return;
    }

    std::vector<double> values;
    double val;
    while (infile >> val) {
        values.push_back(val);
    }
    infile.close();

    if (values.empty()) {
        std::cerr << "ERROR: No data read from
            file." << std::endl;
        return;
    }

    int bins = 50;
    double xmin =
        *std::min_element(values.begin(),
            values.end());
    double xmax =
        *std::max_element(values.begin(),
            values.end());

    TH1F* h = new TH1F("Hist and P_1 values",
        "Normalized Histogram with Poisson_1
        Fit; Time (t); Counts", bins, xmin, xmax);
    for (double x : values) h->Fill(x);

    double bin_width = h->GetBinWidth(1);
    h->Scale(1.0 / (h->Integral() * bin_width));

    h->SetLineColor(kBlue + 2);
    h->SetLineWidth(1);
    h->SetFillColorAlpha(kBlue - 7, 0.4);
    h->SetStats(1);
    h->SetMinimum(0);

    TF1* f = new TF1("pois_1_fit", pois_1,
        xmin, xmax, 1);
    f->SetParameter(0, 0.05);
    f->SetLineColor(kRed + 1);
    f->SetLineWidth(2);

    TCanvas* c = new TCanvas("c_fit", "Poisson
        Fit on Normalized Histogram", 900, 600);
    c->cd();

    h->Draw("E1 BAR");

    h->Fit(f, "R");

    c->Update();
}

#include <TCanvas.h>
#include <TH1F.h>
#include <TF1.h>
#include <TMath.h>
#include <TStyle.h>
#include <fstream>
#include <vector>
#include <iostream>
#include <algorithm>

double pois_0(double* t, double* par) {
    double alpha = par[0];
    return alpha * TMath::Exp(-alpha * t[0]);
}

void fit_normalized_histogram_0() {
    gStyle->SetOptFit(1111);

    const char* filename =
        "D:/root_v6.24.06/macros/x_i_values.txt";
    std::ifstream infile(filename);

    if (!infile.is_open()) {
        std::cerr << "ERROR: Cannot open file "
            << filename << std::endl;
        return;
    }

    std::vector<double> values;
    double val;
    while (infile >> val) {

```

```

        values.push_back(val);
    }
    infile.close();

    if (values.empty()) {
        std::cerr << "ERROR: No data read from
            file." << std::endl;
        return;
    }

    int bins = 50;
    double xmin =
        *std::min_element(values.begin(),
            values.end());
    double xmax =
        *std::max_element(values.begin(),
            values.end());

    TH1F* h = new TH1F("Hist and P_0 values",
        "Normalized Histogram with Poisson_0
        Fit;Time (t);Counts", bins, xmin, xmax);
    for (double x : values) h->Fill(x);

    double bin_width = h->GetBinWidth(1);
    h->Scale(1.0 / (h->Integral() * bin_width));

    h->SetLineColor(kBlue + 2);
    h->SetLineWidth(1);
    h->SetFillColorAlpha(kBlue - 7, 0.4);
    h->SetStats(1);
    h->SetMinimum(0);

    TF1* f = new TF1("pois_0_fit", pois_0,
        xmin, xmax, 1);
    f->SetParameter(0, 0.05);
    f->SetLineColor(kRed + 1);
    f->SetLineWidth(2);

    TCanvas* c = new TCanvas("c_fit", "Poisson
        Fit on Normalized Histogram", 900, 600);
    c->cd();

    h->Draw("E1 BAR");

    h->Fit(f, "R");

    c->Update();
}

#include <TCanvas.h>
#include <TGraph.h>
#include <TAxis.h>
#include <TMath.h>
#include <iostream>

void draw_ratio_graph() {
    double mu[4] = { 138.8, 14.35, 22.53, 2.423
    };
    double sigma[4] = { 11.65, 3.462, 4.58,
        1.752 };
    double ratio[4];

    for (int i = 0; i < 4; ++i) {
        ratio[i] = TMath::Sqrt(mu[i]) / sigma[i];
        std::cout << "mu = " << mu[i] << ",
            sigma = " << sigma[i]
            << ", sqrt(mu)/sigma = " << ratio[i]
            << std::endl;
    }

    TGraph* g = new TGraph(4, mu, ratio);
    g->SetTitle("sqrt(mu) / sigma vs
        mu;#mu;#sqrt{#mu} / #sigma");
    g->SetMarkerStyle(20);
    g->SetMarkerSize(1.2);
    g->SetLineWidth(2);
    g->SetLineColor(kBlue + 2);
    g->SetMarkerColor(kBlue + 2);

    TCanvas* c = new TCanvas("c", "Poisson
        Normality Check", 800, 600);
    g->Draw("AP");

    double xmin = g->GetXaxis()->GetXmin();
    double xmax = g->GetXaxis()->GetXmax();
    TLine* line = new TLine(xmin, 1.0, xmax,
        1.0);
    line->SetLineStyle(2);
    line->SetLineColor(kRed + 1);
    line->SetLineWidth(2);
    line->Draw();
}

#include <TCanvas.h>
#include <TH1F.h>
#include <TF1.h>
#include <TMath.h>
#include <TLegend.h>
#include <fstream>
#include <iostream>

void draw_histogram_with_fits() {
    const char* filename =
        "D:/root_v6.24.06/macros/bal33_ls.txt";
    const char* histname = "bal33_ls";
    int bins = 15;
    double xmin = 0;
    double xmax = 30;

    TH1F* h = new TH1F(histname,
        Form("Histogram of
        %s;Counts;Frequency", histname), bins,
        xmin, xmax);

    std::ifstream infile(filename);
    if (!infile.is_open()) {
        std::cout << "HATA: Dosya acilamadi: "
            << filename << std::endl;
        return;
    }

    double value;
    while (infile >> value) {
        h->Fill(value);
    }
    infile.close();

    double mean = h->GetMean();
    double stddev = h->GetStdDev();
    double peak = h->GetMaximum();

    TCanvas* c = new TCanvas(Form("c_%s",
        histname), histname, 1000, 600);

```

```

h->SetLineColor(kGray+2);
h->SetLineWidth(2);
h->SetStats(0);
h->Draw();

TF1* gaus_fit = new TF1("gaus_fit",
    "[0]*TMath::Gaus(x, [1], [2], true)",
    xmin, xmax);
gaus_fit->SetParameters(peak, mean, stddev);
gaus_fit->SetLineColor(kRed);
gaus_fit->SetLineWidth(2);
h->Fit(gaus_fit, "R+");

TF1* poisson_fit = new TF1("poisson_fit",
    "[0]*TMath::Poisson(x, [1])", xmin,
    xmax);
poisson_fit->SetParameters(peak, mean);
poisson_fit->SetLineColor(kGreen + 2);
poisson_fit->SetLineWidth(2);
h->Fit(poisson_fit, "R+");

gStyle->SetOptFit(0000);

TLegend* legend_gaus = new TLegend(0.62,
    0.70, 0.88, 0.88);
legend_gaus->SetHeader("Gaussian Fit", "C");
legend_gaus->AddEntry(gaus_fit, Form("A =
    %.2f", gaus_fit->GetParameter(0)), "l");
legend_gaus->AddEntry((TObject*)0,
    Form("Mean = %.2f",
    gaus_fit->GetParameter(1)), "");
legend_gaus->AddEntry((TObject*)0,
    Form("Sigma = %.2f",
    gaus_fit->GetParameter(2)), "");
legend_gaus->AddEntry((TObject*)0,
    Form("Chi2/NDF = %.2f",
    gaus_fit->GetChisquare() /
    gaus_fit->GetNDF()), "");
legend_gaus->Draw();

TLegend* legend_pois = new TLegend(0.12,
    0.70, 0.38, 0.88);
legend_pois->SetHeader("Poisson Fit", "C");
legend_pois->AddEntry(poisson_fit, Form("A
    = %.2f", poisson_fit->GetParameter(0)),
    "l");
legend_pois->AddEntry((TObject*)0,
    Form("Lambda = %.2f",
    poisson_fit->GetParameter(1)), "");
legend_pois->AddEntry((TObject*)0,
    Form("Chi2/NDF = %.2f",
    poisson_fit->GetChisquare() /
    poisson_fit->GetNDF()), "");
legend_pois->Draw();

c->Modified();
c->Update();
}

void draw_histogram() {
    const char* filename =
        "D:/root_v6.24.06/macros/ba133_1s.txt";
    const char* histname = "ba133_1s";
    int bins = 10;
    double xmin = 0;

```

```

double xmax = 10;

TH1F* h = new TH1F(histname,
    Form("Histogram of
    %s;Counts;Frequency", histname), bins,
    xmin, xmax);

ifstream infile(filename);
if (!infile.is_open()) {
    cout << "HATA: Dosya acilamadi: " <<
        filename << endl;
    return;
}

double value;
while (infile >> value) {
    h->Fill(value);
}
infile.close();

double mean = h->GetMean();
double stddev = h->GetStdDev();
cout << "Dosya: " << filename << endl;
cout << "Ortalama = " << mean << ",
    StdSapma = " << stddev << endl;

TCanvas* c = new TCanvas(Form("c_%s",
    histname), histname, 800, 600);
h->SetLineColor(kBlue + 2);
h->SetLineWidth(2);
h->SetFillColorAlpha(kAzure + 1, 0.4);
h->Draw();

gStyle->SetOptStat(1111);

c->Update();
}

```

C. FORMULAS

Error Propagation: The general formula for error propagation is given below.

Define a function:

$$y = f(x_1, \dots, x_n) \quad (30)$$

Then the error σ_y of y given as[6]:

$$\sigma_y^2 = \sum_k^n \left(\frac{\partial f}{\partial x_k} \right)^2 \sigma_{x_k}^2 + \sum_{l < k}^m \left(\frac{\partial f}{\partial x_k} \right) \left(\frac{\partial f}{\partial x_l} \right) \sigma_{kl}^2 \quad (31)$$

where σ_{kl} represents the relative error of the variable x_k to x_l . Also, $\sigma_{kl} = \sigma_{lk}$.

If the variables do not depend on each other (relatively free or $\sigma_{kl} = 0$ for all k and l values), equation 31 becomes[6]:

$$\sigma_y^2 = \sum_k^n \left(\frac{\partial f}{\partial x_k} \right)^2 \sigma_{x_k}^2 \quad (32)$$

Weighted Average: We have, say, N different mean values and their uncertainties. We want to find a general mean, which is called a mean of means. In this case, we need to consider

their weights since they not contribute the mean equally. Since they have different uncertainties, their "closeness to the true value" is not the same. Thus, we should calculate their contribution with respect to their weights, which in many cases are errors.[6]

$$\mu_{\mu} \approx \frac{\sum_i^N \frac{\mu_i}{\sigma_i^2}}{\sum_i^N \frac{1}{\sigma_i^2}} \quad (33)$$

and

$$\sigma_{\mu}^2 \approx \frac{1}{\sum_i^N \frac{1}{\sigma_i^2}} \quad (34)$$

REFERENCES

- [1] John R. Taylor. *An Introduction to Error Analysis: The Study of Uncertainties in Physical Measurements*. 2nd ed. University Science Books, 1997.
- [2] Siméon Denis Poisson. *Recherches sur la probabilité des jugements en matière criminelle et en matière civile*. Paris: Bachelier, 1837.
- [3] Ladislaus Bortkiewicz. *Das Gesetz der kleinen Zahlen*. Leipzig: Teubner, 1898.
- [4] Philip R. Bevington and D. Keith Robinson. *Data Reduction and Error Analysis for the Physical Sciences*. 3rd ed. McGraw-Hill Education, 2003.
- [5] E. Gülmez. *Advanced Physics Experiments*. 1st. Boğaziçi University Publications, 1999.
- [6] E. Gülmez. *Basic Data Analysis for Experiments in the Physical Sciences*. Boğaziçi University, 1997.
- [7] SPM Physics. *Geiger-Muller tube*. <http://spmphysics.onlinetuition.com.my/2013/08/geiger-muller-tube.html>. Accessed: 2025-06-05. Aug. 2013.
- [8] Glenn F. Knoll. *Radiation Detection and Measurement*. 4th ed. Wiley, 2010.